



UNIVERSITY OF
TORONTO

Applied Deep Learning

Lecture 5. *Convolutional NNs*

Ali Bereyhi

ECE Department, University of Toronto

Fall 2026



UNIVERSITY OF
TORONTO

Lecture Overview

Recap: Generalization

*When does **overfitting** happen? It happens when*

- 1. **the model** has a **large capacity***
- 2. our training **dataset** is **small***
- 3. parameters start to **co-adapt***

Mutually Related Sources

*For a **large-capacity model**, e.g., **too deep or wide**, training data can be **insufficient** leading to **extreme overfitting**, especially if we **keep training for too many epochs!***

Lecture 5. *Regularization and Convolutional NNs*

Today, we study the following topics

1. Regularization

- What is *regularization*?
- A few well-known regularization schemes, e.g., *weight decay*, *Lasso*, *dropout*

2. Convolutional NNs

- What does *convolution* do?
- What are the *components* of CNNs?
- How can we design a *deep* CNN?



UNIVERSITY OF
TORONTO

Regularization

Training by Penalized Risk: *Regularization*

Regularization

In *training* with *regularization*, we minimize a *penalized (regularized)* form of the empirical risk, i.e.,

$$\min_{\mathbf{w}} \hat{R}(\mathbf{w}) + \Pi(\mathbf{w}) \quad (\text{Regularized Training})$$

$\Pi(\mathbf{w})$ is a penalty that describes the behavior of $\mathbf{w}$ in the case of *overfitting*

Classical Regularization Approaches

Some **well-known** regularization approaches are

- ℓ_2 or **Tikhonov regularization** in which we add a term proportional to $\|\mathbf{w}\|^2$

$$\Pi(\mathbf{w}) = \lambda \|\mathbf{w}\|^2$$

- ↳ This way we **avoid** very **large weights**
- ↳ This **prevents perfect fit** to **training** dataset **reducing** chance of **overfitting**
- ↳ It is also called **weight decay** $\rightsquigarrow$ Check why!

- ℓ_1 or **Lasso regularization** in which we add a term proportional to $\|\mathbf{w}\|_1$

$$\Pi(\mathbf{w}) = \lambda \|\mathbf{w}\|_1 = \lambda \sum_{i=1}^D |w_i|$$

- ↳ This way enforce $\mathbf{w}$ to be **sparse**, i.e., to have to many zeros
- ↳ This way we **reduce** the **capacity of NN** and thus prevent **overfitting**

Regularizing by *Dropout*

A less conventional **regularization** approach is **dropout** that was proposed by Hinton et al. first in *their 2012 paper* and then in *their 2014 paper*¹

for each **training** iteration, we **deactivate** some **nodes** of NN **at random**
or in other words we **drop them out**

Intuition Behind Dropout

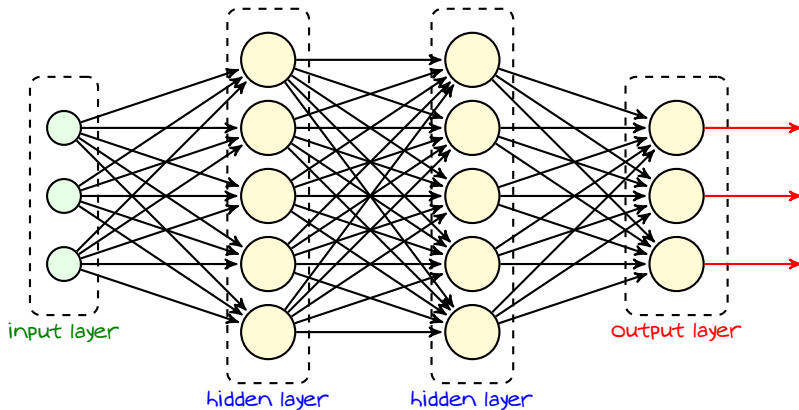
Overfitting means that NN is **larger** than **required** $\rightsquigarrow$ With **dropout**

- we **randomly** switch among **smaller sub-models**
- many of these **smaller models** **do not overfit**
- the NN somehow represents average of these **not-overfitting models**

¹Click to check out the papers!

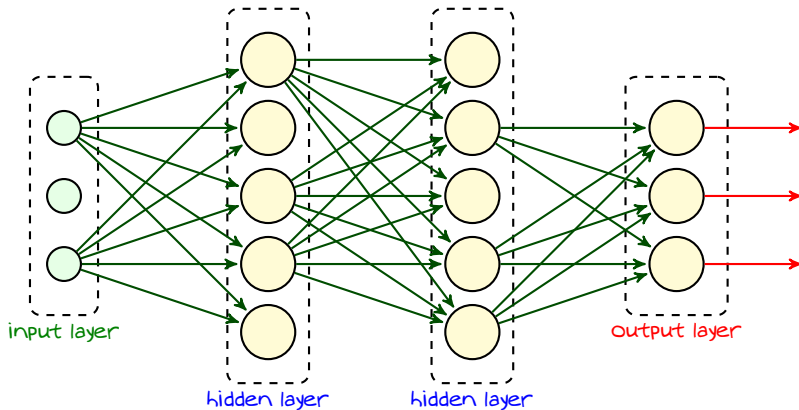
Dropout: *Schematic*

Let's say this is our *NN*



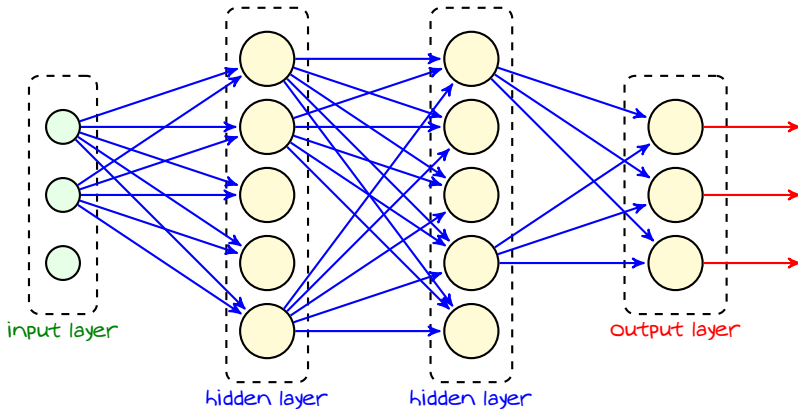
Dropout: Schematic

For *first sample* we select few *nodes* in *each layer*



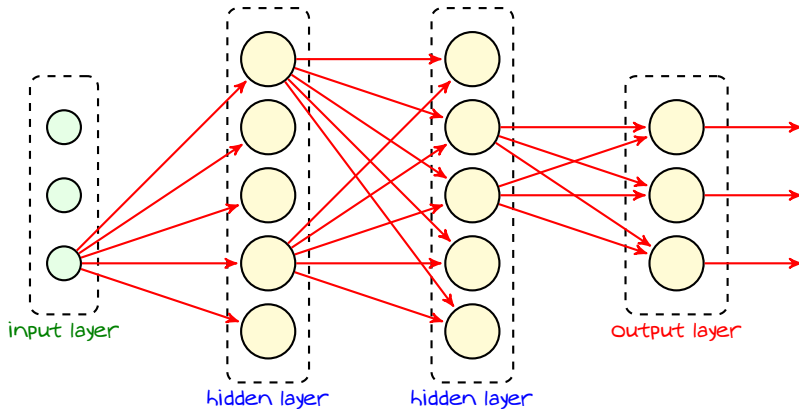
Dropout: Schematic

For *second sample* we select new *nodes* in *each layer* at random



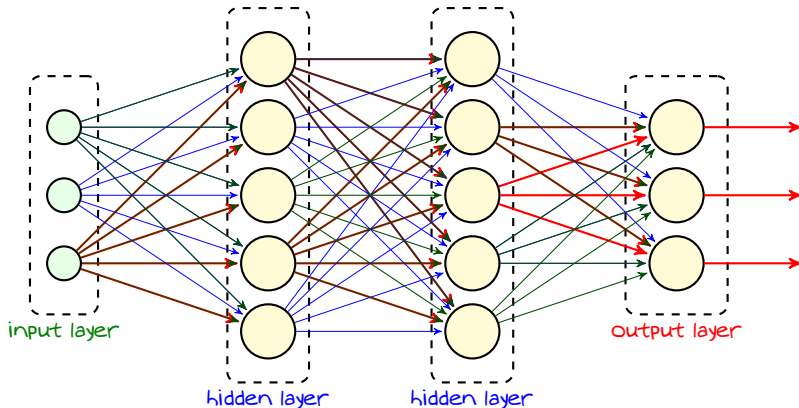
Dropout: Schematic

For *next sample* we select again few *nodes* in *each layer* at random



Dropout: *Schematic*

At the end, we *average* the gradients determined over these *smaller models*



Dropout: *Training Loop*

Training with **dropout** is as before with a **minor change**

For *each sample* pass, we *randomly mask* a *portion of neurons* in each layer

When we pass **forward**, we generate **random masks** for each **layer** $\ell = 0, \dots, L$

- Mask of **layer** ℓ is a **same-size array** with **binary entries**, i.e., $\mathbf{s}_\ell \in \{0, 1\}^{\mathcal{W}_\ell}$
- Entries of $\mathbf{s}_\ell$ are generated **randomly**

$$\text{each entry of } \mathbf{s}_\ell = \begin{cases} 1 & \text{with probability } p_\ell \equiv \text{retention probability} \\ 0 & \text{with probability } 1 - p_\ell \equiv \text{dropout probability} \end{cases}$$

Layer Mask

With **random mask** $\mathbf{s}_\ell = \text{mask}(\mathcal{W}_\ell | p_\ell)$, we can mask layer output $\mathbf{y}_\ell$ as

$$\mathbf{y}_\ell \leftarrow \mathbf{y}_\ell \odot \mathbf{s}_\ell$$

Dropout: *Forward Propagation*

DropoutForwardProp():

```
1: Initiate with  $\mathbf{y}_0 = \mathbf{x}$ 
2: for  $\ell = 0, \dots, L$  do
3:   Generate  $\mathbf{s}_\ell = \text{mask}(\mathcal{W}_\ell | p_\ell)$  # random mask
4:   Set  $\mathbf{y}_\ell = \mathbf{y}_\ell \odot \mathbf{s}_\ell$  # mask output
5:   Add  $y_\ell[0] = 1$  and determine  $\mathbf{z}_{\ell+1} = \mathbf{W}_{\ell+1} \mathbf{y}_\ell$  # forward affine
6:   Determine  $\mathbf{y}_{\ell+1} = f_{\ell+1}(\mathbf{z}_{\ell+1})$  # forward activation
7: end for
8: for  $\ell = 1, \dots, L + 1$  do
9:   Return  $\mathbf{y}_\ell$  and  $\mathbf{z}_\ell$ 
10: end for
```

Backpropagation goes *exactly as before*: of course masked *outputs* participate with value *zero* in gradient computation

Training vs Evaluation

- In *training* we use random mask $\rightsquigarrow$ model.train()
- In *evaluation* we *don't* use random mask $\rightsquigarrow$ model.eval()

Dropout: *Backpropagation*

Two points about dropout to do some research on!

Layer Re-scaling

After training, we *re-scale* weights of each layer with *retention rate*

$$\mathbf{W}_\ell \leftarrow p_\ell \mathbf{W}_\ell$$

A similar approach is included in all classic implementations *but during training*

? Check how *PyTorch* does the re-scaling

Equivalence to Weight Decay

If we apply *dropout at the input layer* of a *linear regression* model, it behaves in expectation like *ℓ_2 -norm regularization*



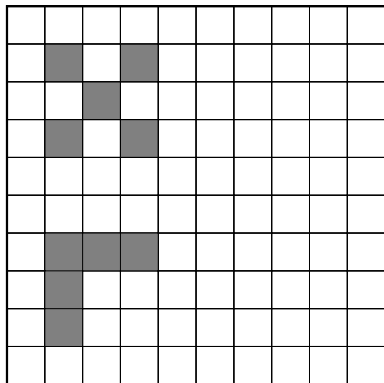
UNIVERSITY OF
TORONTO

Convolutional NNs: *Introduction*

Visual Task: *Pattern Recognition*

? We want to recognize a 3×3 pattern X in a 10×10 grid

Image

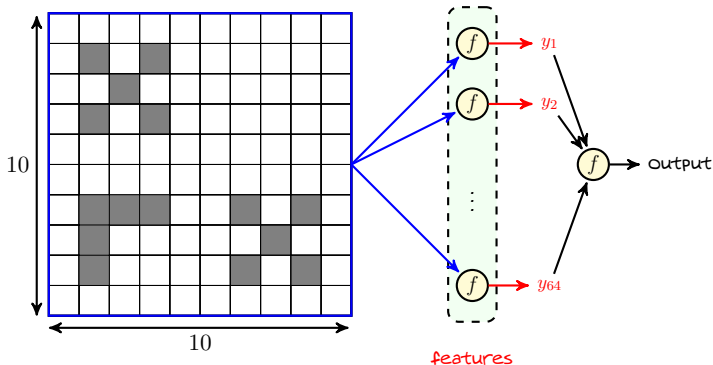


Output

1 $\leftarrow$ X exists
0 $\leftarrow$ X does not exist

Pattern Recognition via MLP

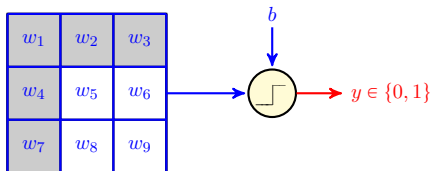
This is **not linear** $\rightsquigarrow$ We go for **an MLP**



$$\# \text{ params} = 64 \times \underbrace{101}_{\# \text{ weights} + \text{bias}} + \underbrace{65}_{\text{output neuron}} = 6,529$$

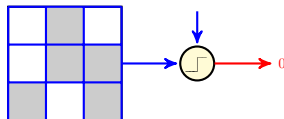
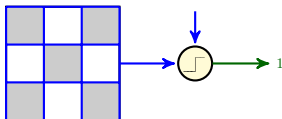
Recall: Classifying X

! Sounds *over-complicated!*



Recall from Assignment 1

We *trained* a perceptron to distinguish between *pattern X* and *any other pattern*

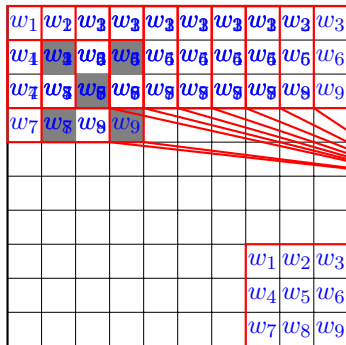


Pattern Recognition via Screening

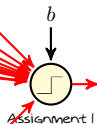
✓ Indeed we can do it *much easier*

$$\# \text{ params} = \underbrace{10}_{\text{same filter}} + \underbrace{65}_{\text{output}} = 75$$

Image X



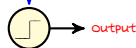
filter W



Feature Y

0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

logical OR



$$Y = \text{Conv}(X|W)$$

Recognizing X

It's enough to have *only a single 1* to recognize "X"

0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

We return the *OR* of all entries in the above *map*

Convolution with Stride 1

image

w_1	w_2	w_3	w_3	w_3	w_3	w_3	w_3	w_3	w_3
w_4	w_5	w_5	w_5	w_5	w_5	w_5	w_5	w_5	w_5
w_7	w_8	w_8	w_8	w_8	w_8	w_8	w_8	w_8	w_8
w_7	w_8	w_8	w_8	w_8	w_8	w_8	w_8	w_8	w_8

filter

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

feature map

$z_{1,1}$	$z_{2,1}$	$z_{3,1}$	$z_{4,1}$	$z_{5,1}$	$z_{6,1}$	$z_{7,1}$	$z_{8,1}$
$z_{1,2}$	$z_{2,2}$	$z_{3,2}$	$z_{4,2}$	$z_{5,2}$	$z_{6,2}$	$z_{7,2}$	$z_{8,2}$
$z_{1,3}$	$z_{2,3}$	$z_{3,3}$	$z_{4,3}$	$z_{5,3}$	$z_{6,3}$	$z_{7,3}$	$z_{8,3}$
$z_{1,4}$	$z_{2,4}$	$z_{3,4}$	$z_{4,4}$	$z_{5,4}$	$z_{6,4}$	$z_{7,4}$	$z_{8,4}$
$z_{1,5}$	$z_{2,5}$	$z_{3,5}$	$z_{4,5}$	$z_{5,5}$	$z_{6,5}$	$z_{7,5}$	$z_{8,5}$
$z_{1,6}$	$z_{2,6}$	$z_{3,6}$	$z_{4,6}$	$z_{5,6}$	$z_{6,6}$	$z_{7,6}$	$z_{8,6}$
$z_{1,7}$	$z_{2,7}$	$z_{3,7}$	$z_{4,7}$	$z_{5,7}$	$z_{6,7}$	$z_{7,7}$	$z_{8,7}$
$z_{1,8}$	$z_{2,8}$	$z_{3,8}$	$z_{4,8}$	$z_{5,8}$	$z_{6,8}$	$z_{7,8}$	$z_{8,8}$

The above operation is **convolution** and we show it as below

$$\text{feature map} = \text{Conv}(\text{image} | \text{filter}, \text{stride} = 1)$$

Convolution with Stride 2

image

w_1	w_2	w_3	w_2	w_3	w_2	w_3	w_2	w_3	
w_4	w_5	w_6	w_5	w_6	w_5	w_6	w_5	w_6	
w_7	w_8	w_9	w_8	w_9	w_8	w_9	w_8	w_9	
w_4	w_5	w_6	w_5	w_6	w_5	w_6	w_5	w_6	
w_7	w_8	w_9	w_8	w_9	w_8	w_9	w_8	w_9	

filter

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

feature map

$z_{1,1}$	$z_{2,1}$	$z_{3,1}$	$z_{4,1}$
$z_{1,2}$	$z_{2,2}$	$z_{3,2}$	$z_{4,2}$
$z_{1,3}$	$z_{2,3}$	$z_{3,3}$	$z_{4,3}$
$z_{1,4}$	$z_{2,4}$	$z_{3,4}$	$z_{4,4}$

We could also play with the *stride* $\equiv$ the step-size by which we move *filter*

$$\text{feature map} = \text{Conv}(\text{image} | \text{filter}, \text{stride} = 2)$$

Convolution with Stride S

Assume $\mathbf{W} \in \mathbb{R}^{F \times F}$ is a **filter**, we also call it **kernel**. Let $\mathbf{X} \in \mathbb{R}^{N \times N}$ be **input map**. We find the output **feature map** as

$$\mathbf{Z} = \text{Conv}(\mathbf{X} | \mathbf{W}, \text{stride} = S)$$

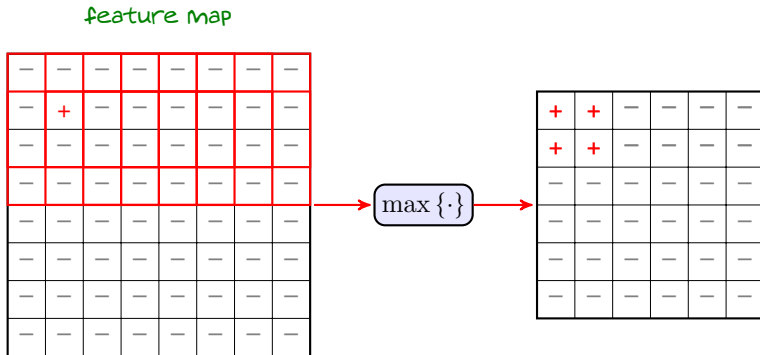
It's enough to find the corresponding sub-matrix for each entry of $\mathbf{Z}$

$$\mathbf{Z}[i, j] = \text{sum}(\mathbf{W} \odot \mathbf{X}_{i,j})$$

where $\mathbf{X}_{i,j}$ is the corresponding $F \times F$ sub-matrix, i.e.,

$$\mathbf{X}_{i,j} = \mathbf{X}[1 + (i - 1)S : F + (i - 1)S, 1 + (j - 1)S : F + (j - 1)S]$$

Pooling: *Max Pooling with Stride 1*



We can **OR** the **pooled feature map**!

- ↳ It looks quite **smoother**!
- ↳ It is of **smaller size**, especially if we **increase the stride**

Convolution: *Capturing Local Features*

- ? Why does convolution compute *efficient features*?
- ✓ Features are stored *locally* $\rightsquigarrow$ i.e. 3×3 *patches* with pattern X
- ➔ Visual patterns in *images* are generally *local*
 - ↳ This is indeed how *visual cortex* processes a scene [Hubel & Wiesel, '59]

J. Physiol. (1959) 148, 574-591

**RECEPTIVE FIELDS OF SINGLE NEURONES IN
THE CAT'S STRIATE CORTEX**

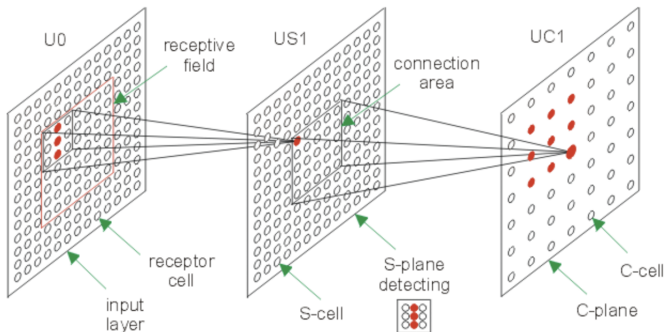
BY D. H. HUBEL* AND T. N. WIESEL*

*From the Wilmer Institute, The Johns Hopkins Hospital and
University, Baltimore, Maryland, U.S.A.*

(Received 22 April 1959)

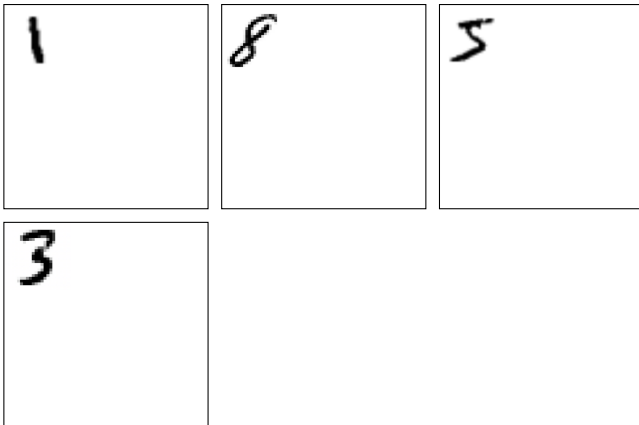
Biological Origin: *Hubel and Wiesel*

This matches the model developed for description of **biological vision**

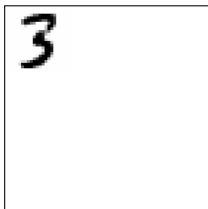


Example II: *MNIST Classification*

Assume *MNIST images* are included in a larger *white background*



MNIST Classification: *MLP Design*



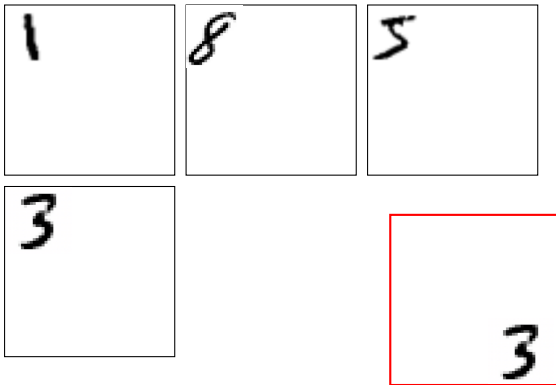
Say the photos with background are *3 times larger* in height and width

- *Images* are now $84 \times 84 \equiv$ we have *7056 pixels*
- *Input layer* has *7056 pixels*

For training, we first *convert* our *MNIST images* into new *larger-size images*

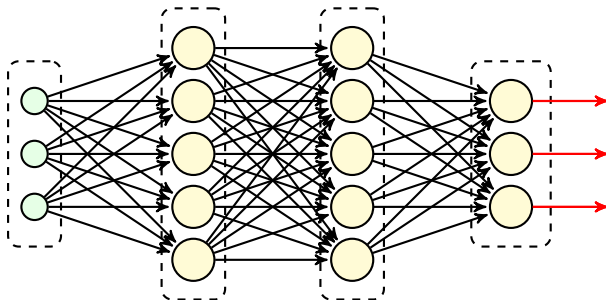
MNIST Classification: *MLP Design*

We do this by *zero-padding*: all *images* after *conversion* lie on *top left corner*



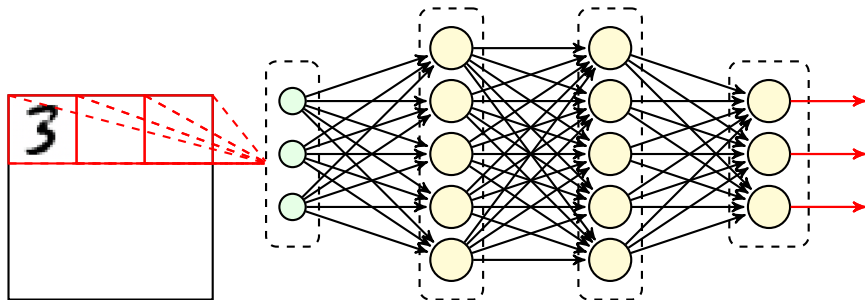
Do you think after training, MLP classifies “3” in *lower right corner* *correctly*? *No!*

MNIST Classification: *Convolutional Design*



- It gets a **784-pixel** image as **input**
- It passes the image through **three fully-connected** layers
- It returns the **class** of the image
 - ↳ If the image **is blank** it returns $\emptyset$

MNIST Classification: *Convolutional Design*



We go through *windows* for size 28×28

1. At each window, we give it *784 pixels* to the FNN to classify
2. We compute *OR* of *outputs* for all *windows*

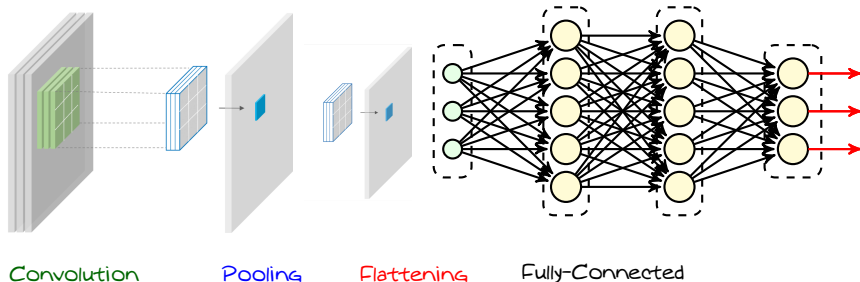
CNNs: *Scanning via Shared Weights*

The given examples are all *simple Convolutional NNs*

- The last *CNN* *extracts features* from the image via a 28×28 *filter*
 - ↳ we learn the *filter's weights*
- The *scanning* procedure has a specific name: *convolution*
 - ↳ it goes through the image by *sliding* over it via a *smaller window*
 - ↳ it computes an *affine transform* of *smaller patches*
- We can look at it as a *giant MLP* with *shared weights*
 - ↳ this affine transform has the *same weights* for many pixels
 - ↳ *not every feature* depends on *every pixels*
 - ↳ it is *not fully-connected*: it's *locally-connected*

CNN: Simple Architecture

Our simple CNN looks like this

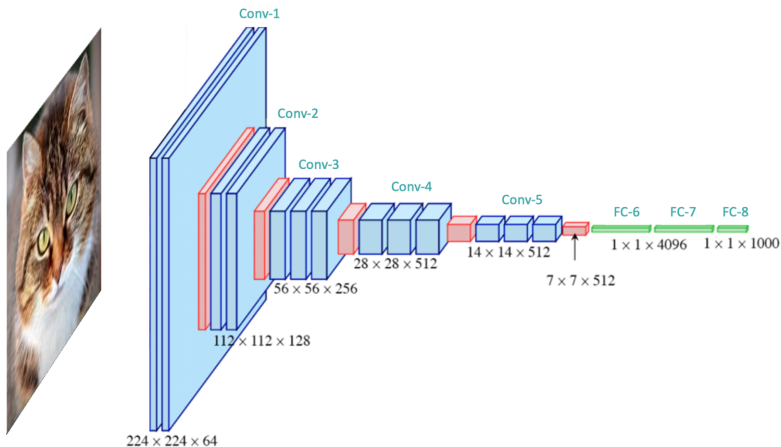


This is a *general architecture* for *CNNs*, but of course we *go deeper!*

- We have more *convolutional* and *pooling layers*
- We do *high-dimensional convolutions* and *more advanced poolings*

CNN: Realistic Architectures

For instance the famous VGG-16 architecture looks like below



Computer Vision: *Short History*

Long study on **biological vision systems** has been conducted

- Hubel and Wiesel studied **visual cortex** in late 1950s
 - ↳ They won **Nobel Prize** in 1981 for their discoveries
- Inspired by that study Fukushima introduced **Neocognitron** in 1979
 - ↳ It was a **convolutional NN** for **unsupervised** learning
- Yann LeCun proposed **LeNet** for supervised learning in 1989
 - ↳ This is pretty much the **beginning of modern CNNs**



UNIVERSITY OF
TORONTO

Components of CNNs

Components of CNNs

CNNs consist of **three** major components

1. **Convolutional** layers

- ↳ they are the **key** component
- ↳ they extract **features** from **input** by sweeping **filters** over **input**

2. **Pooling** layers

- ↳ they **smooth** the **extracted features**

3. **Output FNN**

- ↳ they learn **label** from **final** **extracted features**

Convolution: *Single Map*

2D Convolution

Let $\mathbf{X} \in \mathbb{R}^{N \times M}$ be *input map*: convolution of $\mathbf{X}$ by *filter/kernel* $\mathbf{W} \in \mathbb{R}^{F \times F}$ with stride S is denoted by

$$\mathbf{Z} = \text{Conv}(\mathbf{X} | \mathbf{W}, S)$$

The matrix $\mathbf{Z}$ has $\lfloor (N - F)/S \rfloor + 1$ rows and $\lfloor (M - F)/S \rfloor + 1$ columns and its entry at row i and column j is computed as

$$\mathbf{Z}[i, j] = \text{sum}(\mathbf{W} \odot \mathbf{X}_{i,j})$$

with $\mathbf{X}_{i,j}$ being the corresponding $F \times F$ sub-matrix of $\mathbf{X}$, i.e.,

$$\mathbf{X}_{i,j} = \mathbf{X}[1 + (i - 1)S : F + (i - 1)S, 1 + (j - 1)S : F + (j - 1)S]$$

Convolution: 2D Arrays

Let's see an example: assume $\mathbf{X} \in \mathbb{R}^{4 \times 5}$ and $\mathbf{W} \in \mathbb{R}^{2 \times 2}$

$$\mathbf{X} = \begin{bmatrix} X_{1,1} & X_{1,2} & X_{1,3} & X_{1,4} & X_{1,5} \\ X_{2,1} & X_{2,2} & X_{2,3} & X_{2,4} & X_{2,5} \\ X_{3,1} & X_{3,2} & X_{3,3} & X_{3,4} & X_{3,5} \\ X_{4,1} & X_{4,2} & X_{4,3} & X_{4,4} & X_{4,5} \end{bmatrix} \quad \circledast \quad \mathbf{W} = \begin{bmatrix} W_{1,1} & W_{1,2} \\ W_{2,1} & W_{2,2} \end{bmatrix}$$

$\rightarrow \text{sum}(\mathbf{X}_{1,1} \odot \mathbf{W})$

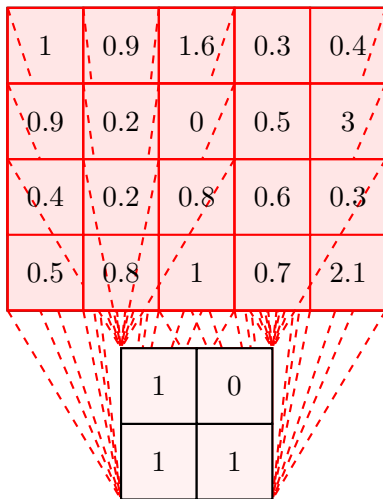
If we convolve with stride $S = 1$

$$\mathbf{Z} = \begin{bmatrix} Z_{1,1} & Z_{1,2} & Z_{1,3} & Z_{1,4} \\ Z_{2,1} & Z_{2,2} & Z_{2,3} & Z_{2,4} \\ Z_{3,1} & Z_{3,2} & Z_{3,3} & Z_{3,4} \end{bmatrix}$$

Let's verify the dimensions of $\mathbf{Z}$

$$\# \text{ rows} = \lfloor (4 - 2)/1 \rfloor + 1 = 3 \quad \# \text{ columns} = \lfloor (5 - 2)/1 \rfloor + 1 = 4$$

Convolution: *Numerical Example*



Convolution: 2D Arrays

Let's see an example: assume $\mathbf{X} \in \mathbb{R}^{4 \times 5}$ and $\mathbf{W} \in \mathbb{R}^{2 \times 2}$

$$\mathbf{X} = \begin{bmatrix} X_{1,1} & X_{1,2} & X_{1,3} & X_{1,4} & X_{1,5} \\ X_{2,1} & X_{2,2} & X_{2,3} & X_{2,4} & X_{2,5} \\ X_{3,1} & X_{3,2} & X_{3,3} & X_{3,4} & X_{3,5} \\ X_{4,1} & X_{4,2} & X_{4,3} & X_{4,4} & X_{4,5} \end{bmatrix} \quad \odot \quad \mathbf{W} = \begin{bmatrix} W_{1,1} & W_{1,2} \\ W_{2,1} & W_{2,2} \end{bmatrix}$$

Diagram illustrating the convolution operation. The input matrix $\mathbf{X}$ is a 4x5 grid. A 2x2 kernel $\mathbf{W}$ is applied to a 2x2 region of $\mathbf{X}$ (indicated by blue dashed lines and boxes). The resulting output element $\mathbf{X}_{1,1}$ is shown, which is the sum of the product of the 2x2 region and $\mathbf{W}$.

Now, we convolve with stride $S = 2$

$$\mathbf{Z} = \begin{bmatrix} Z_{1,1} & Z_{1,2} \\ Z_{2,1} & Z_{2,2} \end{bmatrix}$$

Let's verify the dimensions of $\mathbf{Z}$

$$\# \text{ rows} = \lfloor (4 - 2)/2 \rfloor + 1 = 2 \quad \# \text{ columns} = \lfloor (5 - 2)/2 \rfloor + 1 = 2$$

Convolution: *Numerical Example*

1	0.9	1.6	0.3	0.4
0.9	0.2	0	0.5	3
0.4	0.2	0.8	0.6	0.3
0.5	0.8	1	0.7	2.1

2.1	2.1
1.7	2.5

1	0
1	1

Convolution: *Downsampling*

Let's compare the result with strides 1 and 2

2.1		2.1	
1.7		2.5	

2.1	2.1
1.7	2.5

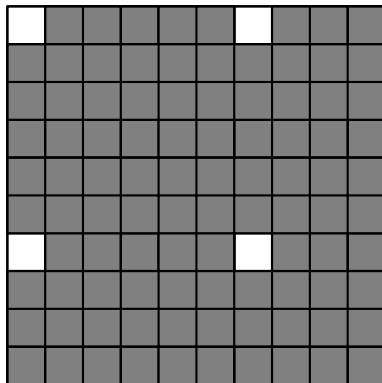
This is *downsampling* with factor 2!

Downsampling

Downsampling with factor f **drops** the last $f - 1$ of *every f columns and rows*

$$\text{dSample}(\mathbf{Z}|f)$$

Convolution: *Downsampling*



Downsampling with **factor 2** $\text{dSample}(\mathbf{Z}|2)$ *Downsampling* with **factor 3**
 $\text{dSample}(\mathbf{Z}|3)$

Convolution: *Downsampling*

Let's compare the result with strides 1 and 2

2.1		2.1	
1.7		2.5	

2.1	2.1
1.7	2.5

Stride as Downsampling

We can look at stride S as *downsampling* with *factor* S , i.e.,

$$\text{Conv}(\mathbf{X}|\mathbf{W}, S) = \text{dSample}(\text{Conv}(\mathbf{X}|\mathbf{W}, 1) | S)$$

Convolution: *Downsampling*

So, we can make an agreement: *by default we consider unit stride, i.e., $S = 1$, i.e., we drop S in convolution from now on*

$$\text{Conv}(\mathbf{X}|\mathbf{W}) = \text{Conv}(\mathbf{X}|\mathbf{W}, S = 1)$$

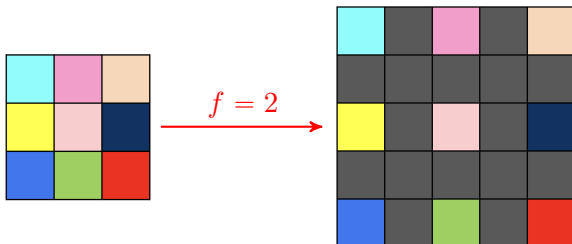
*Whenever we need to convolve with **stride** $S > 1$*

*We add an **downsampling** next to the convolution*

- + *Why do we do this?*
- *We'll see how **easy things** get when we want to **backpropagate**; however, it also helps to **easily** define having a **stride smaller than one**!*

Convolution: *Upsampling*

We can do the *sampling* in other way, i.e., add some zeros



Upsampling

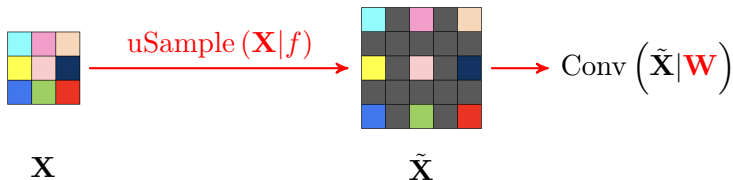
Upsampling with factor f *adds* $f - 1$ rows and columns of *zeros*

$$\text{uSample}(\mathbf{Z}|f)$$

Convolution with Fractional Stride

- + Where do we use *upsampling*?
- If we want to *increase* the size of *the feature map*

To *increase* the size of *the feature map*, we can do following



This is called *fractionally-strided convolution*: for $S < 1$ with $1/S$ being integer

$$\text{Conv}(\mathbf{X}|\mathbf{W}, S) = \text{Conv}(\text{uSample}(\mathbf{X}|1/S)|\mathbf{W})$$

Convolution: *Resampling*

We can extend *stride* to any fraction $S = S_2/S_1$ with integer S_1 and S_2 : we first do *upsampling* with factor S_1 and then *downsampling* with factor S_2

Moral of Story

Convolution with *stride* is equivalent with

convolution with *resampling*

Note that the *order of resampling* differs

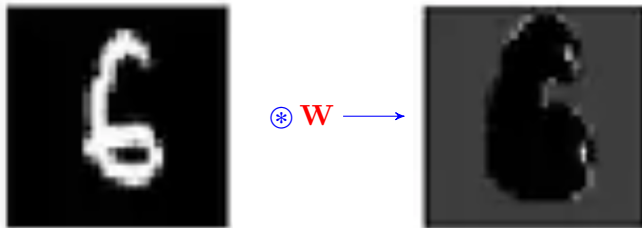
- *Upsampling* is done always *before* convolution
- *Downsampling* is done always *after* convolution

Convolution: *Padding*

In our definition, **feature map Z** has **smaller** dimensions than X even with **stride $S = 1$** . We can play with **dimensions of Z** via **zero-padding**

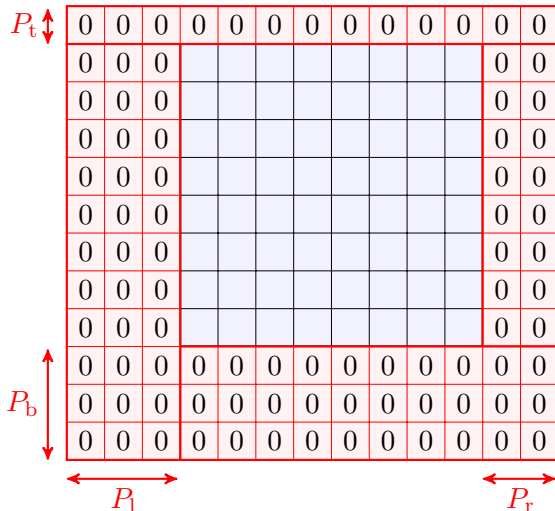
- + Why should we be interested in **changing dimensions of Z** ?
- We'll see **multiple** reasons: a simple one is that we may like to have **a same-size feature map** to sketch it **as an image** and compare it to **input**

For instance in MNIST, we want to sketch the **feature map** as a **28×28 image**



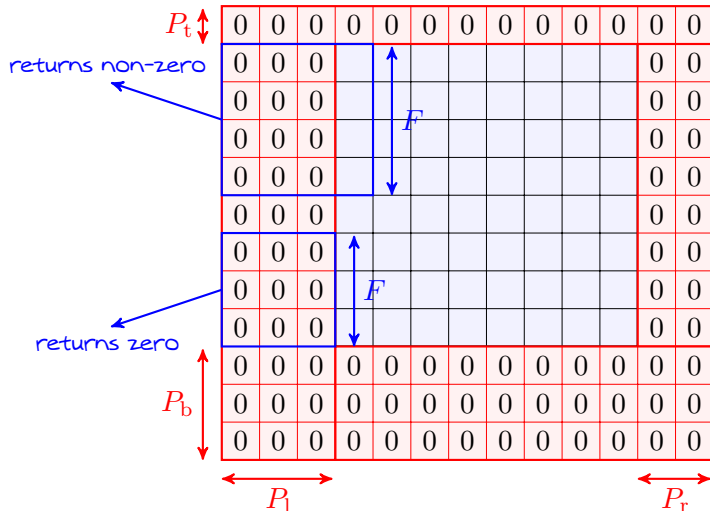
Convolution: Zero-Padding

The trick to *resize feature map* is to *pad zeros at boundaries*



Convolution: Zero-Padding

Typically we *pad* with widths smaller than *filter dimension* F



Convolution: Zero-Padding

With **zero-padding**, the dimensions of **feature map** can be modified: say the **input** has N rows and M columns; then, at the feature map we have

- $\lfloor (N + P_t + P_b - F)/S \rfloor + 1$ rows
- $\lfloor (M + P_l + P_r - F)/S \rfloor + 1$ columns

In practice, we **pad symmetrically**, i.e., $P_b = P_t = P_r = P_l$

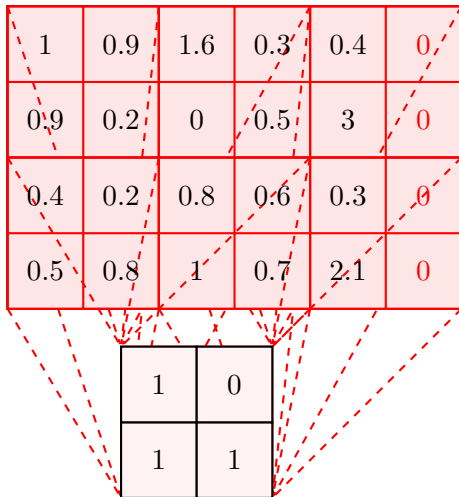
As we see the dimensions of **feature map** is a function of

input dimensions, **stride**, **padding length** and **filter size**

It is thus typical that we get some of these items **not specified**, e.g., we might get **input and output dimensions**, **stride** and **filter size** but not the **padding length**

we can find the **padding length** from other specifications

Example: *Stride = 2 and Padding*



Convolution: *Activation*

Convolution is a *spatial linear transform* on the input image

we should *activate* this *linear transform* if we go *deep*

Convolutional Layer

Convolutional layer with *filter* $\mathbf{W} \in \mathbb{R}^{F \times F}$, *bias* b and *activation function* $f(\cdot)$ transforms the *input map* $\mathbf{X}$ to the *activated feature map* $\mathbf{Y}$ as follows:

1. it first applies *linear convolution* to find $\mathbf{Z}$

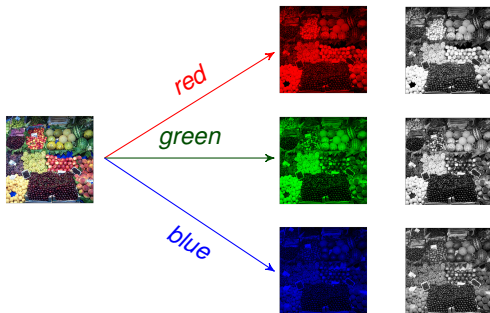
$$\mathbf{Z} = \text{Conv}(\mathbf{X} | \mathbf{W}) + b \quad + \text{applied entry-wise}$$

2. it then *activates* $\mathbf{Z}$

$$\mathbf{Y} = f(\mathbf{Z}) \quad f(\cdot) \text{ applied entry-wise}$$

Convolution: *Multi-Channel Input*

In practice, we have **multi-channel inputs**, for instance *RGB images*



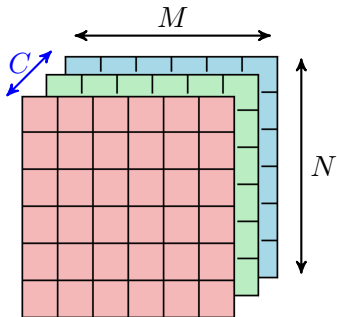
Recap: 3D Tensors

We can think of multi-channel input as a *tensor of order 3*, i.e., a 3D array

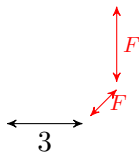
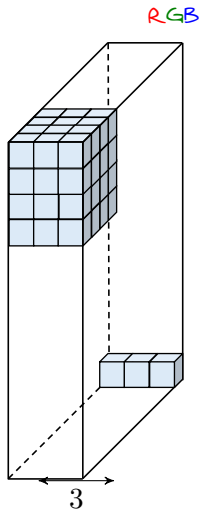
Reminder: Tensor of Order 3

Tensor $\mathbf{X} \in \mathbb{R}^{C \times N \times M}$ is a collection of C matrices each of size $N \times M$

For instance, an $N \times M$ pixel *RGB* image is tensor in $\mathbb{R}^{3 \times N \times M}$

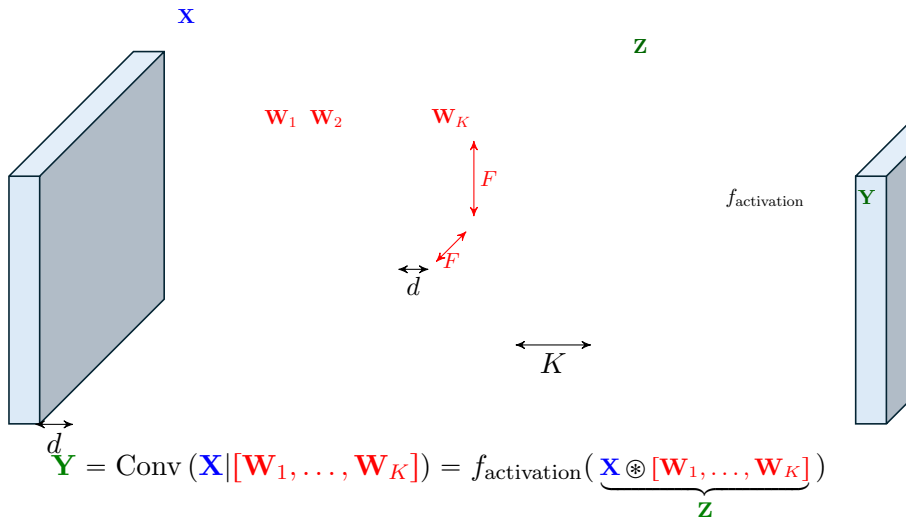


Multi-channel Convolution



✗ # features $\ll$ # input pixels

Multi-channel Convolutional Layers



Convolution Layer: *General Form*

Multi-Channel Convolutional Layer

Convolutional layer with C input channels and K output channels consists of K filters $\mathbf{W}_1, \dots, \mathbf{W}_K \in \mathbb{R}^{C \times F \times F}$, K biases $b_1, \dots, b_K$ and an activation function $f(\cdot)$. It transforms the C -channel input $\mathbf{X}$ to the activated K -channel feature tensor $\mathbf{Y}$ as follows

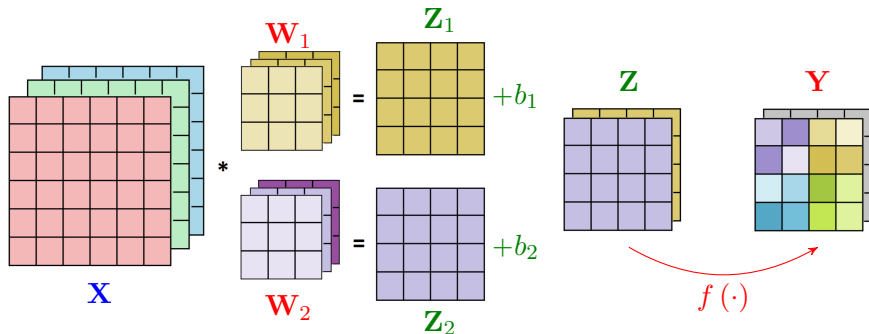
1. it finds the feature tensor $\mathbf{Z}$

$$\mathbf{Z} = [\text{Conv}(\mathbf{X} | \mathbf{W}_1) + b_1, \dots, \text{Conv}(\mathbf{X} | \mathbf{W}_K) + b_K]$$

2. it then activates $\mathbf{Z}$

$$\mathbf{Y} = f(\mathbf{Z}) \quad f(\cdot) \text{ applied entry-wise}$$

Multi-Channel Convolution Layer: Visualization

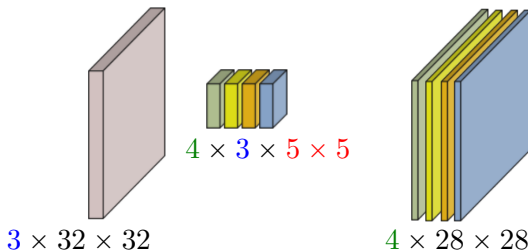


There are few points that we should keep in mind

- **Kernels** have the same **number of channels** (also called **depth**) as **input**
- **Number of kernels** equals the **number of channels** in **feature tensor**
- If X is **output of a convolutional layer** it may have **lots of channels**!
 ↳ We **should not** think that “**input has at most 3 channels**”!

How to Read Dimensions of Convolutional Layers

Many details are dropped as they are readily inferred from architecture



In this diagram, we see that $C = 3$ which is the same in **kernels** and **input**

- We have $K = 4$ **kernels** $\rightsquigarrow$ **feature tensor** has 4 channels
- Kernels** have width $F = 5$ $\rightsquigarrow$ we see that $32 - 5 + 1 = 28$
 - $\hookrightarrow$ **stride** is one, i.e., $S = 1$
 - $\hookrightarrow$ **no zero-padding** is applied $P = 0$

Idea of Pooling: *Smoothing Filters*

The output of convolution can be **jittering** which can come from *spatial correlation*: *Pooling* acts as a filter by sliding over the *extracted features* and pooling out a function of each subpart

- Pooling can **reduce** the **jittering** behavior
- It can mix *extracted features* and potentially improve **shift-invariance**

Shift-Invariance

shift-invariance refers to robustness against simple geometric transform of input

- + How do we do the pooling?
- There are several pooling techniques but popular ones are **max-** and **mean-pooling**

Max-Pooling

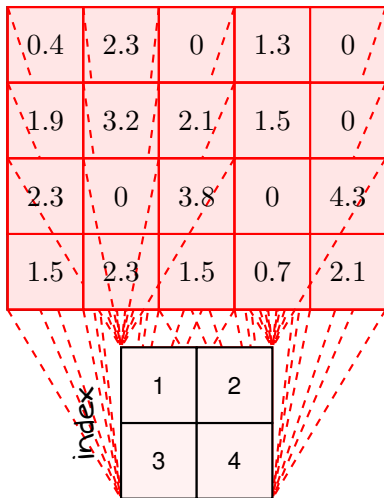
For max-pooling, we use a filter of size $L \times L$ and slide over the **feature map**

$$\mathbf{Y} = \begin{bmatrix} Y_{1,1} & \mathbf{Y_{1,2}} & \cdots & \mathbf{Y_{1,2}} & \cdots & Y_{1,M} \\ Y_{2,1} & Y_{2,2} & \cdots & \cdots & \cdots & Y_{2,M} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ Y_{N,1} & Y_{N,2} & \cdots & \cdots & \cdots & Y_{N,M} \end{bmatrix} \xrightarrow{\text{filter}} \max \{ \mathbf{Y_{1,1}} \mathbf{Y_{1,2}} \mathbf{Y_{i,j}} \}$$

In each window, we pool the **maximum**

$$\hat{\mathbf{Y}} = \begin{bmatrix} \hat{Y}_{1,1} & \hat{Y}_{1,2} & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \end{bmatrix}$$

Max Pooling: Numerical Example



3.2	3.2	2.1	1.5
3.2	3.8	3.8	4.3
2.3	3.8	3.8	4.3

4	3	3	3
2	4	3	4
1	2	1	2

index

Mean-Pooling

In mean-pooling, we again use an $L \times L$ filter and slide over the **feature map**

$$\mathbf{Y} = \begin{bmatrix} Y_{1,1} & \mathbf{Y_{1,2}} & \cdots & \mathbf{Y_{1,2}} & \cdots & Y_{1,M} \\ Y_{2,1} & Y_{2,2} & \cdots & \cdots & \cdots & Y_{2,M} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ Y_{N,1} & Y_{N,2} & \cdots & \cdots & \cdots & Y_{N,M} \end{bmatrix}$$

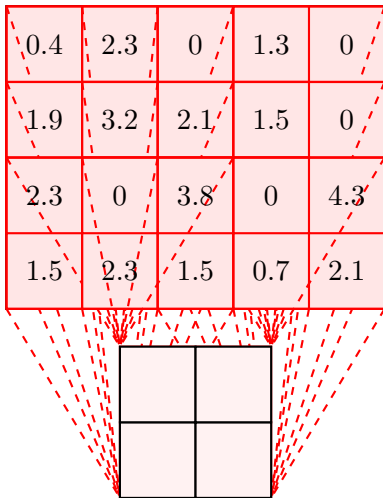
mean $\{ \mathbf{Y_{1,1}} \mathbf{Y_{1,2}} \mathbf{Y_{i,j}} \}$

In each window, we pool the **average**

$$\hat{\mathbf{Y}} = \begin{bmatrix} \hat{Y}_{1,1} & \hat{Y}_{1,2} & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \end{bmatrix}$$

We can look at mean-pooling as a **convolution** with **uniform kernel**

Mean-Pooling: *Numerical Example*



2.05	1.9	1.225	0.7
1.85	2.275	1.85	1.45
1.525	1.9	1.5	1.775

Advanced Pooling: Use a General Function

We could in general replace **max** or **mean operator** with a general function

$$\mathbf{Y} = \begin{bmatrix} Y_{1,1} & Y_{1,2} & \dots & Y_{1,M} \\ Y_{2,1} & Y_{2,2} & \dots & Y_{2,M} \\ \vdots & \vdots & \vdots & \vdots \\ Y_{N,1} & Y_{N,2} & \dots & Y_{N,M} \end{bmatrix} \xrightarrow{\Pi(\cdot)} \mathbb{R}^{L \times L} \mapsto \mathbb{R}$$

Typical functions are **norms**, e.g., ℓ_2 -norm

$$\hat{\mathbf{Y}} = \begin{bmatrix} \hat{Y}_{1,1} & \hat{Y}_{1,2} & \dots & \dots \\ \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \end{bmatrix}$$

We can even replace it with a **small NN**!

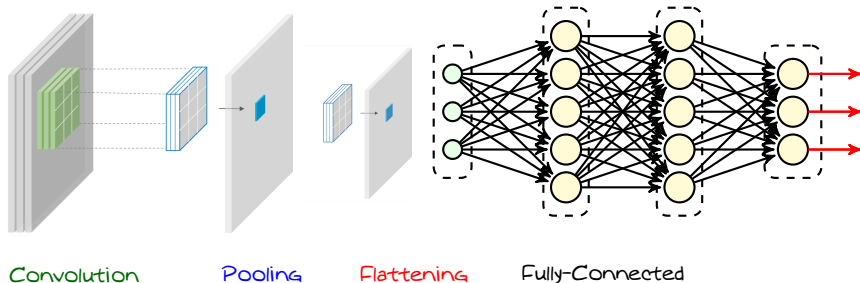
Pooling with Stride

Similar to convolution, we can apply *pooling* with *stride*

- We usually do not use *fractional stride* with *pooling*
 - ↳ Extra zeros usually do not make any gain: *think of max-pooling for instance*
- Integer *stride* can be seen as downsampling
- In practice, we often leave integer *strides* for *pooling* layer
 - ↳ We apply convolution with *unit stride*
 - ↳ If we need to down-sample, we do it later in the *pooling* layer
- In many architectures *pooling* is applied *every couple of convolutional layers*
 - ↳ We apply *multiple* convolutional layers
 - ↳ We then apply a *pooling* layer *with stride*

Output MLP: *Flattening*

Let's recall our simple CNN

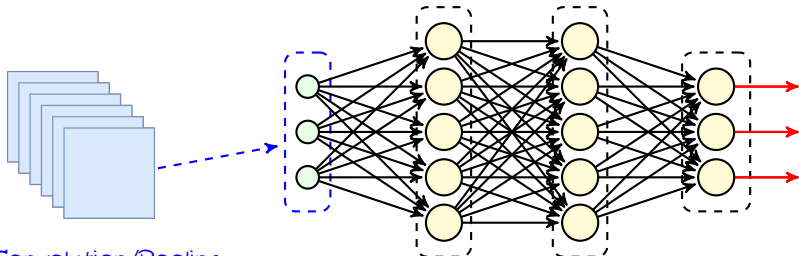


Once we are over with convolution and pooling we *flatten* final *feature tensor*

Flattening

*In **flattening**, we sort all entries of the **feature tensor** into a vector*

Flattening



Last Convolution/Pooling

Say we have a *feature tensor* with K channels

↳ each channel is an $N \times M$ map after last *convolution* or *pooling*

We then have an input to the FNN with NMK entries

After *flattening* everything goes as before through the *MLP*

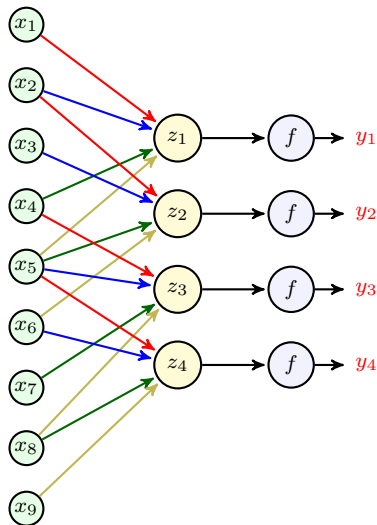
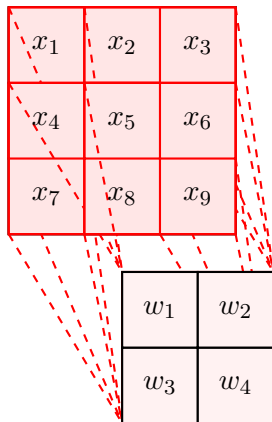


UNIVERSITY OF
TORONTO

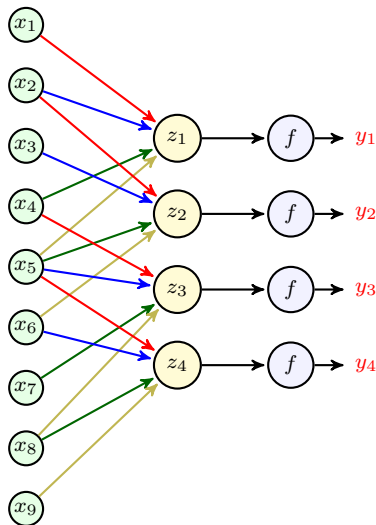
Components of CNNs

- Local Connectivity

Convolutional Layer as *Set of Neurons*



Convolutional Layer as Set of Neurons



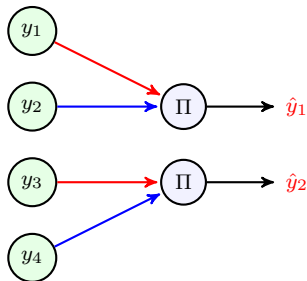
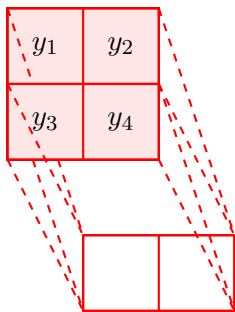
In this viewpoint

- ↳ each **feature** is output of a neuron
- ↳ neurons **share** same weights and bias
- ↳ neurons are **activated by** $f(\cdot)$

*This is however not a **fully**-connected layer*

- ↳ it is **locally**-connected
- ↳ neurons have **shared parameters**

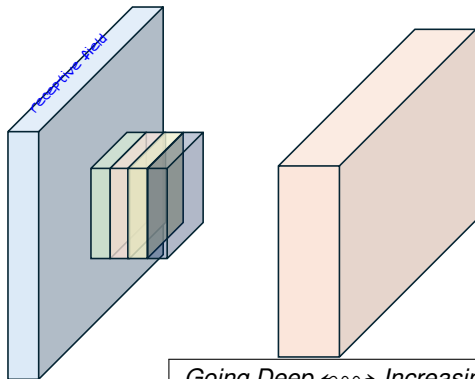
Pooling Layer as Set of Neurons



Pooling as a Layer

Similar to convolutional layer, pooling is a *feedforward layer* with *local connectivity* and *shared* parameters

Going Deep: *From Local to Global Features*



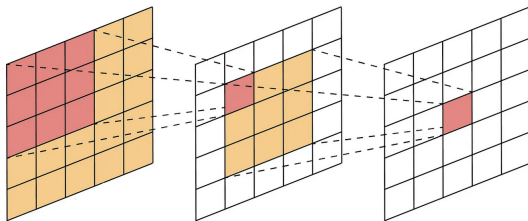
Going Deep $\longleftrightarrow$ Increasing *Receptive Field*

- ? But, we end up *too much global* features!
- ✓ Well, we may use *pooling* or *strided convolution*

Going Deep: *Receptive Field*

As we go **deep** in CNN, we mix **features**: features in **deeper** layers depend on more pixels of input tensor. We often say

*deeper layers have larger **receptive field***



Receptive Field

The **input region** that each neuron in **a given convolutional layer** responds to

General Perspective on Deep CNNs

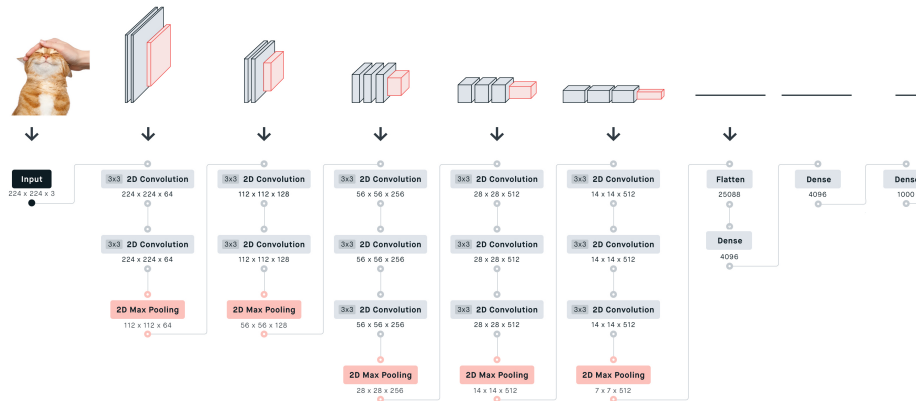
The given example illustrates an *intuitive interpretation* of *deep* CNNs

- ↳ Deep CNNs extract *gradually* the *features* of input
- ↳ In first layers they extract features of *smaller patches* of the input
- ↳ They *gradually* expand this region as they go *deeper*
 - ↳ *deeper* layers look at *high-level features*

This gives us a good idea on *how* a good *deep* CNN looks like

- First layers focus on *understanding local features*
 - ↳ the computed of *feature maps* are *large*
 - ↳ the number of *output channels* is *small*
- As we go *deeper* in the CNN, we get more *global understanding*
 - ↳ the computed of *feature maps* are *small*
 - ↳ the number of *output channels* *increases*

Deep CNN: VGG-16



Deep CNN: *Example*

Let's count the number of layers with *learnable parameters*

# layers with weights =	2	first two convolutions
	+ 2	second two convolutions
	+ 3	third round of convolutions
	+ 3	fourth round of convolutions
	+ 3	fifth round of convolutions
	+ 3	fully-connected network
	= 16	

This is actually why it's called VGG-16



UNIVERSITY OF
TORONTO

Further Read

Further Read

Read More on These Topics



Goodfellow, et al., *Deep Learning*

↳ [Chapter 9](#)



Bishop & Bishop, *Deep Learning: Foundations and Concepts*

↳ [Chapter 10](#)

Nice to Check

1. *Receptive fields, Binocular Interaction and Functional Architecture in the Cat's Visual Cortex*
↳ Hubel and T. Wiesel (1962)
2. *Gradient-Based Learning Applied to Document Recognition*
↳ Y. LeCun et al. (1998)